OS Orange problem

Sudarshan G
Section H
PES1UG24AM450

Phase 1:

Screenshot 1A:

```
(base) sudarshan@MacBookAir PES1UG24AM450-pes-vcs % ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B:

```
All Phase 1 tests passed.
(base) sudarshan@MacBookAir PES1UG24AM450-pes-vcs % find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

Phase 2:

Screenshot 2A:

```
make: `test_tree' is up to date.
(base) sudarshan@MacBookAir PES1UG24AM450-pes-vcs % ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B:

```
sudo: .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf: command not found
(base) sudarshan@MacBookAir PES1UG24AM450-pes-vcs % sudo xxd .pes/objects/2a/594d39232787fba8eb7287418aec99
c8fc2ecdaf5aaf2e650eda471e566fcf | head -20
Password:
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
(base) sudarshan@MacBookAir PES1UG24AM450-pes-vcs %
```

Phase 3:

Screenshot 3A:

```
(base) sudarshan@MacBookAir PES1UG24AM450-pes-vcs % ./pes add file1.txt f
ile2.txt
zsh: segmentation fault  ./pes add file1.txt file2.txt
(base) sudarshan@MacBookAir PES1UG24AM450-pes-vcs % ./pes status
Staged changes:
  (nothing to show)

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  file2.txt
  untracked:  index.h
  untracked:  file1.txt
  untracked:  object.c
  untracked:  pes.c
  untracked:  Makefile
  untracked:  test_objects
  untracked:  tree.h
  untracked:  test_objects.c
  untracked:  commit.c
  untracked:  test_tree
  untracked:  README.md
  untracked:  index.c
  untracked:  test_sequence.sh
  untracked:  .gitignore
  untracked:  dump_tree.c
  untracked:  test_tree.c
  untracked:  pes.h
  untracked:  commit.h
  untracked:  tree.c
```

Phase 5 and 6

Q 5.1)

We must first update .pes/HEAD so it points to refs/heads/<branch>, effectively switching the active branch reference. Then, we need to load the commit hash stored in that branch and retrieve its tree from the object store. The index must be replaced with this new tree so it reflects the exact snapshot of the target commit. Finally, the working directory must be transformed to match this tree by creating new files, deleting obsolete ones, and modifying existing ones. The complexity comes from doing this safely and efficiently without overwriting uncommitted user changes, handling nested directories, and ensuring consistency between the index, object store, and filesystem.

Q 5.2)

To detect a dirty working directory conflict, we rely on comparing the working directory, index, and target commit without any shortcuts. For each tracked file, compute its hash from the working directory and compare it with the hash stored in the index; if they differ, the file has uncommitted changes. Then compare the index version with the version in the target commit's tree. If the file differs between the index and the target commit, it means the checkout would modify that file. A conflict occurs when both conditions are true simultaneously: the user has modified the file locally and the target branch also has a different version In this case, checkout must refuse to proceed to prevent overwriting changes that are not yet committed.

Q 5.3)

In a detached HEAD state, HEAD points directly to a commit hash instead of a branch reference, meaning new commits are created without being attached to any branch. These commits exist in the object graph but are not reachable from any named reference, so they are floating. If the user switches to another branch, these commits lose visibility and may be garbage collected. However, they are not immediately lost as Git maintains a reflog that records recent HEAD positions. By inspecting the reflog, the user can find the commit hash of these detached commits and create a new branch pointing to them, thereby restoring them into the normal branch structure and preventing their deletion.

Q 6.1)

To clean up unreachable objects, we run a graph traversal starting from all roots, which are branch heads, tags etc, and then mark everything reachable from them. From each root commit, you recursively visit its parent commits and associated tree objects, and from each tree you visit blobs

and subtrees, building a full set of reachable object hashes. The most efficient data structure for tracking this is a hash set, since we need constant-time checks to avoid revisiting objects and to mark reachability cleanly. After traversal, we iterate through all objects in the object store and delete anything not present in the reachable set. For a repository with 100,000 commits and 50 branches, we're not multiplying 100k by 50, because branches share history heavily. We visit roughly on the order of 100k commits along with their associated trees and blobs, around a few hundred thousand total objects depending on file churn.

Q 6.2)

Running garbage collection concurrently with a commit operation will invite a race condition where one part of the system deletes objects that another part hasn't finished linking yet. For example, in case of a commit in progress, it writes blobs and trees to the object store, but hasn't yet updated a branch reference to point to the new commit. At the same moment, GC runs, scans reachable objects starting from existing branch heads, and doesn't see these newly written objects as reachable yet, so it deletes them. The commit process then tries to finalize by creating a commit object referencing trees or blobs that no longer exist, corrupting the repository. Git avoids this mess through mechanisms like write ordering and reachability guarantees, including writing objects first, using temporary references, and relying on reflogs and grace periods so that recently created objects are not immediately eligible for deletion. It also uses locking and avoids aggressive deletion by deferring actual cleanup, ensuring that anything in flight doesn't get wiped out just because GC showed up at the worst possible time.